Pretraining Knowledge Base — Schema and Retrieval Contract

This repository follows the source → wiki → generated-query structure used by KernelWiki, adapted to scientific pretraining. Existing project instructions are in AGENTS.md. This file specifies knowledge representation and retrieval.

Navigation

```
python scripts/kb.py query "muP batch" --tag optimization
python scripts/kb.py get technique-hyperparameter-transfer --follow-sources
python scripts/kb.py claim claim-marin-transfer-d1536
python scripts/kb.py questions
python scripts/kb.py status
```

Start at index.md. Use queries/ to navigate by topic, problem, stage, source, evidence and reproducibility. The existing long-form doc/ guides remain companions, not discarded material.

Literature and the three knowledge layers

The literature library is retained. `references/` contains the downloaded papers, LaTeX, figures, code archives, report snapshots and result records. Do not replace it with summaries. `knowledge/literature-baseline.json` records the pre-existing library so validation detects missing originals.

1. `sources/`: one YAML-frontmatter source card per paper/report/repository, relevant PR, issue or HF release. The card points into the original library and preserves version, retrieval date and artifact hashes. Ingestion alone gives `review_status: catalogued`, not a scientific audit.
2. `wiki/`: canonical synthesis pages, cross-referenced by stable IDs. Each carries type, tags, confidence, reproducibility, prerequisites, related pages, source IDs and claim IDs. Numerical comparisons bind to complete claim records.
3. `queries/`: generated indices. Regenerate; do not edit manually.

`knowledge/` holds the schemas, vocabulary, aliases, claims, version registry, question contracts, small replay inputs and execution receipts. It is named separately from the ignored `data/` corpus directory to avoid mixing schemas with training data. `doc/` remains the detailed reading/experiment manual.

Page types and required fields

The executable contract is knowledge/schemas.yaml. Every canonical source/wiki page has `id`, `title`, `type`, controlled `tags`, `architectures`, and `sources` IDs. Architecture labels describe the discussion scope; claim-level model scope determines where empirical evidence applies.

The original `doc/` guides are indexed separately by `knowledge/legacy-guides.json` as catalogued companions. They are not silently promoted to schema-reviewed wiki pages.

Type	ID prefix	Additional required fields
source-doc	doc-	URL, category, version, retrieval date, local artifacts, review status
source-pr	pr-	repository/PR, author, captured status, merged flag, head SHA, artifacts
source-dataset	dataset-	HF/source revision, category, retrieval date, artifact hashes

Type	ID prefix	Additional required fields
source-model	model-	model revision, category, retrieval date, artifact hashes; weight-access scope
source-issue	issue-	captured experiment record, version, date and artifact hashes
wiki-foundation	foundation-	confidence, reproducibility/scope, prerequisites, related IDs, claims
wiki-technique	technique-	foundation fields plus actual code examples; at least snippet
wiki-pattern	pattern-	symptoms, candidate techniques, related/prerequisite/source/claim IDs
wiki-case-study	case-	code examples and fully specified performance claims
wiki-recipe	recipe-	runnable/snippet scope, code examples and acceptance boundary
wiki-governance	governance-	evidence policy, scope, related/source/claim IDs

Confidence is distinct from reproduction

- `verified`: both authoritative-document and upstream-implementation evidence, named in `evidence_basis.official_sources` and `.upstream_code`. It does **not** mean the model was retrained or the numerical result independently replicated.
- `source-reported`: at least one authoritative source supports the account.
- `inferred`: our synthesis, derivation or operational recommendation.
- `experimental`: a proposed or insufficiently validated recipe.

Reproducibility is `concept → pseudocode → snippet → runnable → benchmarked`. Technique, recipe and case-study pages require at least a syntax-valid referenced snippet. Every page states `reproducibility_scope`: an executable corpus-record check is not an executable model-training recipe. A `benchmarked` claim needs an execution receipt and environment/metric details. Current CPU receipts cannot certify GPU training reproduction.

Claim-level verification is separately recorded as `source_read`, `artifact_inspected`, `cpu_replayed` or `training_reproduced`. Do not promote a paper's reported result because we downloaded its PDF or checked its hash.

Numerical and performance claims

Claims live in knowledge/claims.json. Each records kind, sources and exact locators, model/data/optimizer/horizon/evaluation scope, measurements with units, limits, version dependencies and verification status. Comparisons must identify the baseline and uncertainty; `not reported` is an explicit limitation, not permission to invent an interval.

```yaml
performance_claims:
  - claim_id: claim-marin-transfer-d1536
    metric: macro_bpb
    value: 0.91844
    unit: bits/byte
    model: d1536 mixture-swap ladder rung
    data: three-phase Harrier-store experiment; see frozen issue
    compute: 90767 updates; not a measured runtime saving
    optimizer: recipe linked in issue; not independently qualified here
    evaluation: final Paloma 16-subset macro
    baseline: old ladder in issue 8435
    uncertainty: no final-rung confidence interval retrieved
```

```
    source_id: issue-marin-9126
    source_locator: issue body and completion comment 5688777500
```

Stored, routed-active and executed parameters; unique tokens and repeated exposures; sequence and token batch; macro and pooled BPB; fitted compute savings and measured runtime must remain separate quantities. Paper coefficients retain their source units and conditional recipe.

Quality gates

```
python scripts/kb_workbench.py --all --check
python scripts/kb.py validate --verify-artifacts --check-generated
python -m unittest discover -s tests -p 'test_kb*.py'
python scripts/kb.py generate-indices
```

Validation enforces IDs and references, tags/aliases, prerequisites, source pins, claim scope, confidence prerequisites, receipt-supported CPU checks, bidirectional version pointers, question coverage contracts, artifact hashes and preservation of the original literature. These checks catch structural/provenance regressions; they do not establish scientific truth or complete corpus-wide semantic review.

Source snapshots are appendable. Preserve historical versions and distinguish Harrier/Luxical generations, code configurations, proposed experiments, measured finals, infrastructure smokes and independently reproduced results. No GPU/cloud job is authorized merely by reading a runnable command in this knowledge base.